Microsoft Defender for Identity

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Microsoft Defender for Identity 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 6

Boundary . . . 6

Effort . . . 6

1. Solution Overview — Microsoft Defender for Identity 6

What Microsoft Defender for Identity is . . . 6

Why this matters . . . 7

How it works — the three phases . . . 7

What you can control . . . 8

Limitations (be honest with buyers) . . . 8

Connections and authentication used . . . 9

The standard deployment model . . . 9

What is out of scope (and where it lives) . . . 9

2. Requirements Guide — Microsoft Defender for Identity 10

2.1 Defender for Identity activated in the portal . . . 10

2.2 Domain controllers reachable and supported for the sensor . . . 10

2.3 Directory Service Account (DSA) — read-only, gMSA recommended . . . 10

2.4 A pilot domain controller chosen . . . 10

2.5 Sensitive entities identified for tagging . . . 11

2.6 Honeytoken accounts planned . . . 11

2.7 Identity posture (ISPM) scope agreed . . . 11

2.8 Advanced-hunting read access (identity tables) . . . 11

2.9 A test workstation and test account for benign PoC signal . . . 11

2.10 Evaluation criteria agreed for the PoC . . . 12

2.11 Facilitator and attendee roles for the enablement labs . . . 12

2.12 Connected response workloads and governance . . . 12

Checklist . . . 12

Global Requirements — Defender (category-level) 13

G1. Microsoft 365 / Entra tenant . . . 13

G2. Defender licensing . . . 13

G3. Administrative roles (least privilege) . . . 13

G4. Onboarding reach to endpoints / identity / workloads . . . 13

G5. Network access to Defender service endpoints . . . 14

G6. A pilot ring . . . 14
G7. Log Analytics workspace (where required) . . . 14
G8. Response governance . . . 14
3. Implementation Guide — Microsoft Defender for Identity 14
Naming (set once, reuse verbatim) . . . 15
Phase A — Deploy (onboard the pillar) 15
Step 1 — Assess readiness . . . 15
Step 2 — Activate MDI and prepare the Directory Service Account . . . 15
Step 3 — Install the pilot sensor (pilot DC) . . . 16
Step 4 — Tag sensitive entities (Tier-0) . . . 16
Step 5 — Plant and tag honeytokens . . . 16
Step 6 — Enable and triage identity posture (ISPM) . . . 16
Step 7 — Validate detections on the pilot . . . 17
Phase B — Enable the team (five hands-on labs) 17
Lab 0 — Readiness check . . . 17
Lab 1 — Identity attack paths & the entity page . . . 17
Lab 2 — Recon & lateral-movement detections . . . 17
Lab 3 — Honeytokens . . . 18
Lab 4 — ISPM posture (read and prioritize, no enforcement) . . . 18
Lab 5 — Investigation end-to-end + response rehearsal . . . 18
Phase C — Prove the value (guided PoC) 19
Step 8 — Generate benign test signal . . . 19
Step 9 — Validate, demonstrate, and score . . . 19
Step 10 — Tear down the PoC-only scope . . . 19
Expand (across the fleet) 20
Step 11 — Expand to remaining DCs and AD FS/AD CS . . . 20
5. Testing Guide — Microsoft Defender for Identity 20
Part A — Deploy validation (Phase A) . . . 20
Part B — Team enablement (Phase B knowledge-check) . . . 21
Part C — PoC validation (Phase C) . . . 22
6. Troubleshooting Guide — Microsoft Defender for Identity 23
DSA not working . . . 23
Sensor unhealthy . . . 23
No identity signal . . . 24
Detection does not fire . . . 24
No ISPM recommendations . . . 24

Honeytoken generating noise . . . 24
Honeytoken candidate list empty (`signInActivity` blank) . . . 24
Entity pages / advanced hunting empty during the labs . . . 25
Response actions greyed out (Lab 5) . . . 25
Recon signal not generated (PoC) . . . 25
No reconnaissance alert (PoC) . . . 25
Teardown left residue (PoC) . . . 25
Response action acted without approval . . . 25
Script cannot authenticate . . . 25
Portal menu paths don't match . . . 26
7. Best Practices — Microsoft Defender for Identity 26
Deploy in a ring, never all at once . . . 26
Protect and monitor the sensors . . . 26
Use a gMSA for the Directory Service Account . . . 26
Honeytokens are your highest-signal detection . . . 26
Tag Tier-0 so alerts are weighted correctly . . . 27
Work ISPM as a trend, not a one-off . . . 27
Enable the team on a pilot, read-only by default . . . 27
Prove value with benign signal only — never real attacks . . . 27
Keep response actions approval-gated until trusted . . . 27
Keep licensing and preview facts current . . . 27
Common anti-patterns to avoid . . . 28
Where to go next . . . 28
8. Deliverables Summary — Microsoft Defender for Identity 28
Documentation (source/) . . . 28
Automation assets (scripts/) . . . 29
Take-home artefacts (produced during the engagement) . . . 31
What the buyer receives . . . 31
Outcome . . . 31
Appendix A — PowerShell Scripts 33

Executive Summary — Microsoft Defender for Identity

White-label · Defender

Identity is the top attack surface — almost every intrusion that reaches an enterprise moves *through* Active Directory. Yet Microsoft Defender for Identity (MDI), the **identity threat detection & response (ITDR)** pillar of Microsoft Defender, is routinely left half-finished: a sensor on one domain controller, a partly-configured Directory Service Account, no honeytokens, un-triaged identity posture (ISPM), a team that has never investigated a real identity alert, and value nobody has ever proven to the business. The console looks "on," but it sees a fraction of AD and no one is confident driving it.

This is the **complete Microsoft Defender for Identity solution** — one product that takes MDI end to end across its whole lifecycle:

1. **Deploy it right** — install sensors on every domain controller (and AD FS / AD CS where present), configure the read-only Directory Service Account (DSA), tag sensitive Tier-0 entities, plant and tag honeytokens, turn on and triage identity posture (ISPM), and validate that detections actually fire — piloted on one DC first, then expanded.

2. **Enable the team** — five hands-on labs that take your admins and SOC analysts from "we have it" to "we operate it": reading identity attack paths, recognizing recon and lateral-movement detections, deploying honeytokens, working ISPM, and running a clean, approval-gated investigation — with a runbook and KQL pack they keep.

3. **Prove the value** — a guided, in-tenant proof of concept that generates **benign, simulated** signal (harmless read-only directory enumeration plus a honeytoken touch — never a real attack), demonstrates the headline ITDR capabilities on that signal, scores the result against your criteria, and cleanly tears the PoC scope down.

Written for a competent admin with no prior MDI experience, this is a **deploy-it-today** package. Everything disruptive is **pilot-first** and **approval-gated**; response actions stay approval-gated throughout; and no existing AD control is ever removed to "test."

What you get

• A plain-language **Solution Overview** of MDI — sensors, the DSA, honeytokens, sensitive tags, ISPM posture, detections, the operator console, and how a safe PoC proves it.

• A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (category-wide ones live in `global-requirements-defender.md`).

• An **Implementation Guide** that is one continuous playbook: deploy the pilot sensor → tag entities and plant honeytokens → enable/triage ISPM → run the five enablement labs → run the guided PoC → expand across the fleet.

• Six **automation assets**: read-only discovery, validation, and posture/scorecard reporting (Microsoft Graph Security), a `-WhatIf`-gated deploy/rollback helper (sensitive-entity and honeytoken tagging), a **benign** PoC test-signal generator, and a **KQL advanced-hunting pack** for the identity tables.

• **Testing**, **Troubleshooting**, and **Best-Practices + where-to-go-next** guides for a confident rollout.

What this delivers

A **fully sensored, team-ready, value-proven** Defender for Identity delivering ITDR — sensors healthy on every domain controller, the DSA configured, honeytokens planted, Tier-0 tagged, ISPM triaged, detections proven to fire, a team that can investigate unaided, and a scorecard the business can stand behind — the whole lifecycle in one package, not a half-configured console.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess readiness (license, roles, DCs, DSA), configure the DSA and portal settings, pilot one DC (sensor, tags, honeytokens), validate that detections fire and the team can operate, then expand sensor-by-sensor with response kept approval-gated until trusted.

Boundary

This solution deploys, enables, and proves the **Defender for Identity pillar itself**. It is distinct from the unified **Microsoft Defender XDR — Unified Onboarding** solution (which brings all four pillars into `security.microsoft.com`), the **SOC Operations — Sentinel + Defender XDR** solution, and the **Advanced Threat Hunting (KQL)** solution — cross-reference those, do not duplicate them. Incident-response, automated-response guardrail, regulatory, and licensing determinations remain the customer's security, operations, and compliance functions; this guide provides the technical configuration and the operating skill.

Effort

A competent administrator can complete the pilot-DC deployment and detection validation in a focused engagement, run the five enablement labs in a one- to two-day session, and complete the guided PoC on the same pilot DC — then expand across the remaining domain controllers on a controlled change cadence. Sensor rollout scales with domain-controller count and change-window availability.

1. Solution Overview — Microsoft Defender for Identity

What Microsoft Defender for Identity is

Microsoft Defender for Identity (MDI) is the **identity threat detection & response (ITDR)** pillar of Microsoft Defender. It detects attacks against **on-premises Active Directory** — and the identity infrastructure around it (AD FS, AD CS, Entra Connect) — by installing a lightweight **sensor** directly on domain controllers and federation/certificate servers. The sensor inspects local authentication and directory traffic and behavior, streams signal to the cloud service, and surfaces alerts in the unified Defender portal at `https://security.microsoft.com` under **Identities**. MDI catches the identity phase of an intrusion:

- **Reconnaissance** — account/SPN/network enumeration (LDAP/SAMR/DNS recon), the classic first step of an attack (MITRE Discovery, TA0007).
- **Compromised credentials** — brute force and password spray (Credential Access, TA0006).
- **Lateral movement** — Pass-the-Hash, Pass-the-Ticket, overpass-the-hash, remote code execution (Lateral Movement, TA0008).

• **Domain dominance** — DCSync, DCShadow, Golden Ticket, skeleton key (Credential Access / Persistence).

Because the sensor runs *on* the domain controller, there is **no agentless option** for on-prem AD — deploying MDI is fundamentally a **sensor-deployment** exercise, done in a change window, one server at a time.

MDI also surfaces **Identity Security Posture Management (ISPM)** recommendations (exploitable misconfigurations such as unconstrained delegation, legacy protocols, printer-spooler on DCs, risky certificate templates), lets you plant **honeytokens** (decoy accounts that should never authenticate — any touch is high-fidelity evidence of an intruder), and feeds **Defender XDR**: MDI alerts correlate into unified incidents, enrich the **user/device entity pages** with a timeline and **UEBA / investigation-priority** scoring, and are queryable in **advanced hunting** (the `Identity*` tables).

Why this matters

Almost every attack that reaches an enterprise passes through Active Directory. Attackers who land on one endpoint pivot to AD to escalate, and traditional endpoint or SIEM tooling barely sees that identity movement. MDI is designed exactly for it — but only if it is **fully deployed, operated by a capable team, and proven to work**:

- A tenant with one sensor on one DC has blind spots on every other DC.
- A tenant with no **honeytokens** loses the highest-signal, lowest-noise detection MDI produces.
- A tenant that never triages **ISPM** leaves exploitable posture gaps in place.
- A team that has never investigated an identity alert cannot act on one when it matters.
- A deployment nobody has ever *proven* is a hard sell to the business.

This solution closes all five gaps — the whole pillar deployed, the team enabled, and the value proven — rather than assumed-working.

How it works — the three phases

Phase 1 — Deploy (onboard the pillar)

1. **Directory Service Account (DSA)** — a read-only account (gMSA recommended), `MDI Directory Service Account – Tenant – Prod`, is prepared so sensors can query AD for extra context, and registered under **Settings** → **Identities** → **Directory Service accounts**.
2. **Sensors** are installed on **domain controllers** first (then AD FS / AD CS), each registered with an **access key** from the portal, and confirmed **Running / Healthy**.
3. **Sensitive entities** (Tier-0 — Domain/Enterprise Admins, DCs, AD FS/AD CS, PKI, backup, break-glass) are **tagged** as `MDI Sensitive Entities – Tier0 – Prod` so MDI weights alerts on them correctly.
4. **Honeytokens** — decoy accounts `MDI Honeytoken – Decoy Identity – Pilot` — are planted and tagged.
5. **ISPM** recommendations are enabled and triaged into a remediation shortlist.

6. **Detections are validated** — a benign, controlled test confirms an alert fires in the portal and in advanced hunting.

Phase 2 — Enable the team (five hands-on labs)

Run against the pilot/lab scope, read-only or preview-first, each with a pass/fail gate:

• **Lab 1 — Identity attack paths & the entity page.** Read a user/device entity page; interpret investigation priority and lateral-movement paths.

• **Lab 2 — Recon & lateral-movement detections.** Recognize the detection families, map them to MITRE ATT&CK, triage an alert; run and save the recon/lateral-movement hunting queries.

• **Lab 3 — Honeytokens.** Choose, tag, and verify a honeytoken; understand and monitor its detection.

• **Lab 4 — ISPM posture.** Read and prioritize the recommendations into a shortlist (no enforcement).

• **Lab 5 — Investigation end-to-end.** Run a full identity investigation across the entity timeline and advanced hunting; document findings; rehearse the **approval-gated** response.

Phase 3 — Prove the value (guided PoC)

1. **Generate benign signal** — harmless read-only enumeration (`net group` / `nltest` / LDAP reads — the same commands admins run daily) plus a honeytoken touch. **No real attack, exploit, credential theft, or malware, ever.**

2. **Demonstrate** the signal produces a real detection — an alert in the incident queue, an entry on the identity timeline, rows in advanced hunting, and UEBA scoring.

3. **Score** the observations against your written evaluation criteria.

4. **Tear down** the PoC-only scope cleanly (untag, resolve test alerts) — the production sensors remain.

What you can control

• **Sensor coverage** — which DCs and AD FS/AD CS servers carry a sensor, and the order they onboard (pilot DC first).

• **Directory Service Account** — a read-only gMSA (recommended) versus a standard account.

• **Sensitive-entity tagging** and **honeytokens** — which entities are high-value and which decoys are tripwires.

• **ISPM triage** — which posture recommendations you remediate first.

• **Which capabilities to prove** in the PoC and the pass/fail bar.

• **Response posture** — approval-gated throughout; automation decisions stay with the SOC.

Limitations (be honest with buyers)

• MDI is **license-gated** (Defender for Identity, commonly via M365 E5 / E5 Security or a standalone plan). Deployment without the plan fails — *verify current availability in your tenant.*

• **Sensors run on domain controllers** and need resources, network reach to the MDI service, and a **change window** — not a portal-only toggle, and not installable agentlessly.

• MDI protects **on-premises AD**; cloud-only (Entra ID) identity risk is covered by Entra ID Protection and adjacent solutions, not MDI.

• **Detections, tables, ISPM, and UEBA** populate over time — allow warm-up after each sensor comes online before judging coverage.

• PoC signal is **benign and simulated only** — it proves the detection pipeline, not resilience to a real adversary (that is a red-team engagement, out of scope).

• MDI improves identity detection; it does not replace AD hardening, patching, tiered administration, or the SOC's incident-response decisions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com` → **Settings** → **Identities**) for activation, DSA, sensors, tags, honeytokens, ISPM, incidents, entity pages, and advanced hunting.

• **Microsoft Graph Security** and the advanced-hunting API (via the `Microsoft.Graph` PowerShell modules) for read-only discovery, validation, reporting, scoring, and hunting over `IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `IdentityQueryEvents`, and `AlertInfo`. *Confirm exact scopes and availability in your tenant.*

• **On-premises Active Directory** read helpers (`Get-ADDomainController`, `Get-ADUser`, LDAP/`net`/`nltest`) for sensor-candidate/honeytoken discovery and benign PoC signal.

• Least-privilege roles: **Security Administrator** for configuration (and the honeytoken tag), **Security Reader** for read-only work, **Domain Admin** (change window) for sensor install, and **Global Administrator** only for one-time first-service activation.

The standard deployment model

> ***Assess readiness (license, roles, DCs, DSA) → configure the DSA and portal settings → pilot one DC (sensor, tags, honeytokens) and validate detections → enable the team with the labs and prove value with the PoC → expand to the remaining DCs and AD FS/AD CS.*** *Repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand****, with response actions kept approval-gated until trusted.*

What is out of scope (and where it lives)

• **Unified multi-pillar onboarding** (Endpoint, Office 365, Cloud Apps into one correlated portal) — the **Microsoft Defender XDR — Unified Onboarding** solution.

• **SIEM integration and SOC process design** — the **SOC Operations — Sentinel + Defender XDR** solution.

• **Broad cross-pillar KQL hunting** beyond the identity tables — the **Advanced Threat Hunting (KQL)** solution.

• **Incident-response, regulatory, and records decisions** — the customer's SOC/security/compliance functions.

2. Requirements Guide — Microsoft Defender for Identity

Verify **every** requirement before configuring. Each entry: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, Log Analytics workspace, response governance) are covered once in `global-requirements-defender.md` — read that first; the items below are **specific to deploying, operating, and proving Defender for Identity**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Defender for Identity activated in the portal

Description: MDI initialized for the tenant under `security.microsoft.com` → **Settings** → **Identities**. **Why required:** The workspace, sensor download, and access key only appear after first-time activation; ITDR sensors, detections, honeytokens, and the identity timeline are the capabilities being deployed and proven. **How to verify:** Sign in as a Security Administrator and confirm **Settings** → **Identities** loads with a workspace name. Admin center → **Billing** → **Licenses** shows Defender for Identity. Run `MDIFULL-Discovery.ps1`. *Verify current plan availability in your tenant.* **How to implement if missing:** Assign the Defender for Identity plan (often bundled in M365 E5 / E5 Security) and complete the one-time MDI activation (Global Administrator required for first-service activation only).

2.2 Domain controllers reachable and supported for the sensor

Description: Domain controllers (and AD FS / AD CS servers where present) that meet the current MDI sensor OS/resource requirements and can reach the MDI service. **Why required:** The sensor runs *on* each DC — there is no agentless AD option; unsupported OS or blocked reach means no coverage. **How to verify:** `MDIFULL-Discovery.ps1` lists every DC (`Get-ADDomainController`), its OS, and reachability hints. Confirm outbound HTTPS to the MDI service URLs. *Verify current sensor OS support and endpoint list in your tenant.* **How to implement if missing:** Bring the DC OS to a supported level, confirm resources (CPU/RAM/disk), and schedule a change window per DC.

2.3 Directory Service Account (DSA) — read-only, gMSA recommended

Description: A read-only **Directory Service Account** the sensors use to query AD for extra context. Current guidance recommends a **group Managed Service Account (gMSA)**, named `MDI Directory Service Account – Tenant – Prod`. **Why required:** Without the DSA, several detections and enrichment are limited; a gMSA avoids a static password. **How to verify:** Confirm the gMSA is created and installable on the sensor hosts. `MDIFULL-Discovery.ps1` reports DSA readiness. *Verify current MDI DSA requirements.* **How to implement if missing:** Create the gMSA `MDI Directory Service Account – Tenant – Prod`, grant the sensor hosts permission to retrieve its password, and register it under **Settings** → **Identities** → **Directory Service accounts**.

2.4 A pilot domain controller chosen

Description: One domain controller selected as `MDI Sensor Host – Pilot DC – Pilot` to carry the first sensor and to host the enablement labs and the PoC. **Why required:** Deployment is validated on one DC before the rest (per `global-requirements-defender.md`); a bad rollout on every DC at once is avoidable, and a single pilot DC keeps the PoC scope reversible. **How to verify:** The pilot DC is named and change-approved; `MDIFULL-Discovery.ps1` flags it. **How to implement if missing:** Pick a healthy, representative (or non-critical / lab) DC and reserve a change window before installing the pilot sensor.

2.5 Sensitive entities identified for tagging

Description: The high-value accounts, groups, and servers (Tier-0 — Domain Admins, Enterprise Admins, DCs, AD FS/AD CS, PKI, backup, break-glass) to be **tagged sensitive** as `MDI Sensitive Entities – Tier0 – Prod`. **Why required:** MDI weights alerts on tagged sensitive entities more strongly; untagged Tier-0 dilutes signal. **How to verify:** A Tier-0 inventory exists; `MDIFULL-Validate.ps1 -Check Sensitive` reports which entities are already tagged. **How to implement if missing:** Build the Tier-0 list and tag under **Settings** → **Identities** → **Entity tags** → **Sensitive** (or preview with `MDIFULL-Deploy.ps1 -Mode SensitiveTag`).

2.6 Honeytoken accounts planned

Description: Dormant, unused decoy accounts — `MDI Honeytoken – Decoy Identity – Pilot` — that no legitimate process ever authenticates or queries. **Why required:** Honeytokens are the highest-confidence, lowest-noise MDI detection: any touch is suspicious by definition, which also makes them the crisp trigger for the PoC. **How to verify:** The decoy accounts exist in AD and are unused (no recent sign-ins); `MDIFULL-Discovery.ps1` lists candidate no-activity accounts (read-only) and `MDIFULL-Validate.ps1 -Check Honeytoken` confirms tagging. **How to implement if missing:** Create idle decoy accounts with believable names (e.g. `svc-legacy-backup` in `contoso.com`), ensure nothing uses them, then tag them under **Settings** → **Identities** → **Entity tags** → **Honeytoken**. *Do not tag a real, in-use account.*

2.7 Identity posture (ISPM) scope agreed

Description: Agreement to enable and triage **Identity Security Posture Management (ISPM)** recommendations surfaced by MDI into Microsoft Secure Score / Exposure management. **Why required:** ISPM finds exploitable AD/identity misconfigurations (unconstrained delegation, spooler on DCs, legacy protocols, risky certificate templates); ignoring them leaves attack surface open. Lab 4 works these recommendations. **How to verify:** `security.microsoft.com` → **Secure Score** → **Recommended actions** (Identity) shows MDI recommendations after sensors are healthy. `MDIFULL-Report.ps1` lists them. *Verify current ISPM availability; assessments populate over hours to a few days after sensors report.* **How to implement if missing:** Confirm sensors are healthy (ISPM depends on sensor data), allow warm-up, then review the Identity recommendations and assign owners.

2.8 Advanced-hunting read access (identity tables)

Description: Operators can open **Advanced hunting** in the Defender portal and via the Graph Security API (`ThreatHunting.Read.All`) query the `Identity*` tables. **Why required:** Detection validation, the enablement labs, and the PoC scorecard confirm the signal landed in `IdentityInfo`, `IdentityLogonEvents`, `IdentityQueryEvents`, `IdentityDirectoryEvents`, and `AlertInfo`. **How to verify: Hunting** → **Advanced hunting** → run `IdentityInfo | take 5`; confirm rows. `MDIFULL-Validate.ps1` returns rows. **How to implement if missing:** Grant the running account **Security Reader** (or higher) plus hunting permission, and consent to the read-only hunting scope.

2.9 A test workstation and test account for benign PoC signal

Description: A non-privileged, domain-joined **test workstation** and a **test account** from which to run harmless read-only directory enumeration for the PoC. **Why required:** The benign recon signal (LDAP/`net`/`nltest`) must originate from a real client so the sensor observes it; a dedicated test account keeps the signal attributable and the teardown clean. **How to verify:** You can sign in to a domain-joined test workstation with a test account and run `net group` / `nltest` queries. **How to implement if missing:** Provision a test workstation and a low-privilege test account before the PoC phase.

2.10 Evaluation criteria agreed for the PoC

Description: A short, written list of what "pass" means — e.g. *the reconnaissance alert fires within N minutes, appears on the identity timeline, is queryable in hunting, and the honeytoken activity is flagged.* **Why required:** A PoC is only conclusive if the bar is set **before** you run it; the scorecard maps observations to these criteria. **How to verify:** A criteria list (5–8 items) exists; `MDIFULL-Report.ps1 -Scorecard` accepts it as input. **How to implement if missing:** Draft the criteria with the evaluating stakeholders before the PoC phase. *What counts as "good enough" is the customer's decision.*

2.11 Facilitator and attendee roles for the enablement labs

Description: One **facilitator** (Security Administrator) to drive the single configuration action (tagging the honeytoken); **attendees** with **Security Reader** (plus hunting) to follow along read-only. **Why required:** Least privilege — only the facilitator needs a configuration action; everyone else learns read-only and rehearses response in a **simulate / approval-gated** posture. **How to verify:** Entra → **Roles and administrators** — confirm the facilitator holds Security Administrator and attendees hold Security Reader. **How to implement if missing:** Assign the roles before the session; keep write/response roles reserved for the approval-gated response rehearsal only.

2.12 Connected response workloads and governance

Description: For MDI response actions (disable user, force password reset), the on-prem/hybrid identity workloads must be connected, and a response-governance position agreed. **Why required:** Response actions are disruptive; the organization decides the guardrails (per `global-requirements-defender.md`). **How to verify:** A documented response policy exists; actions are approval-gated. `MDIFULL-Validate.ps1` / `MDIFULL-Report.ps1` report the current configuration. **How to implement if missing:** Keep response actions approval-gated at first; *incident-response and automation-guardrail decisions are the SOC/security team's — this guide covers the technical configuration.*

Checklist

- MDI activated in the portal — `MDIFULL-Discovery.ps1`
- Domain controllers supported/reachable for the sensor
- DSA (gMSA) `MDI Directory Service Account – Tenant – Prod` prepared
- Pilot DC `MDI Sensor Host – Pilot DC – Pilot` chosen + change window
- Sensitive entities `MDI Sensitive Entities – Tier0 – Prod` identified
- Honeytoken accounts `MDI Honeytoken – Decoy Identity – Pilot` planned

- ISPM scope agreed — `MDIFULL-Report.ps1`
- Advanced-hunting read access (`ThreatHunting.Read.All`)
- Test workstation + test account for benign PoC signal
- PoC evaluation criteria written — `MDIFULL-Report.ps1 -Scorecard`
- Facilitator (Sec Admin) + attendees (Sec Reader) for the labs
- Response governance (approval-gated) documented

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints

are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Microsoft Defender for Identity

Follow these steps **in order**. Most configuration is in the **unified Defender portal** (`https://security.microsoft.com` → **Settings** → **Identities**); the sensor installs on the domain controllers themselves. This is one continuous playbook across three phases: **Phase A — Deploy** (onboard the pillar), **Phase B — Enable the team** (five labs), **Phase C — Prove the value** (guided PoC), then **Expand**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Prepare the DSA, install the sensor on* ***one pilot domain controller*** *first, tag entities and plant honeytokens,* ***validate that a detection actually fires****, enable the team on the pilot, prove the value with benign signal, and only then expand to the remaining DCs and AD FS/AD CS. Keep* ***response actions*** *approval-gated throughout. Never remove an existing AD control to "test."*

Naming (set once, reuse verbatim)

- Directory Service Account (gMSA): `MDI Directory Service Account – Tenant – Prod`
- Pilot sensor host / DC: `MDI Sensor Host – Pilot DC – Pilot`
- Sensitive-entity tag scope: `MDI Sensitive Entities – Tier0 – Prod`
- Honeytoken decoy account(s): `MDI Honeytoken – Decoy Identity – Pilot`
- Detection-validation saved query: `MDI Detection Validation – Tenant – Validate`
- Saved hunting queries (labs): `Recon – Identity – Hunt`, `LateralMovement – Identity – Hunt`, `Honeytoken – Identity – Hunt`, `Investigation – Identity – Hunt`
- ISPM remediation shortlist: `MDI ISPM Shortlist – Tier0 – Prod`
- Investigation findings doc: `MDI Investigation – Identity – Validate`
- PoC benign-signal tag: `MDI-TESTSIGNAL`
- PoC scorecard output: `MDI PoC Scorecard – Pilot DC – Validate`

*Portal navigation paths reflect the current Microsoft Defender portal. Microsoft moves menus periodically — *verify current navigation in your tenant*; the Troubleshooting guide lists moved-menu fallbacks (use global search for a feature name).*

Phase A — Deploy (onboard the pillar)

Step 1 — Assess readiness

1. Sign in to `security.microsoft.com` as a **Security Administrator**.
2. Run `MDIFULL-Discovery.ps1` to capture current state: MDI activation/workspace, licensed-plan hints, admin roles, every domain controller and its OS, DSA readiness, existing sensor status, and candidate no-activity honeytoken accounts. Changes nothing.
3. Choose the pilot DC and record it as `MDI Sensor Host – Pilot DC – Pilot`. Reserve a change window.

Step 2 — Activate MDI and prepare the Directory Service Account

1. `security.microsoft.com` → **Settings** → **Identities**. If MDI is not initialized, complete the one-time activation (Global Administrator, first time only) and note the **workspace** name.
2. Create the read-only gMSA `MDI Directory Service Account – Tenant – Prod` in AD, and grant the sensor host computer accounts permission to retrieve its password.
3. **Settings** → **Identities** → **Directory Service accounts** → **Add credentials** — register the gMSA. *Verify current MDI DSA requirements.*
4. Run `MDIFULL-Validate.ps1 -Check DSA` to confirm the DSA is registered.

Step 3 — Install the pilot sensor (pilot DC)

1. **Settings** → **Identities** → **Sensors** → **Add sensor** — download the sensor package and copy the **access key**.

2. During the approved change window, copy the package to `MDI Sensor Host – Pilot DC – Pilot`, run the installer, and supply the **access key** when prompted.

3. Confirm outbound HTTPS from the DC to the MDI service URLs (per `global-requirements-defender.md`). *Verify current endpoint list.*

4. **Settings** → **Identities** → **Sensors** — confirm `MDI Sensor Host – Pilot DC – Pilot` shows **Running / Healthy**.

5. Run `MDIFULL-Validate.ps1 -Check Sensor` to confirm the pilot sensor is healthy and identity signal is flowing.

Step 4 — Tag sensitive entities (Tier-0)

1. `security.microsoft.com` → **Settings** → **Identities** → **Entity tags** → **Sensitive**.

2. Add the Tier-0 accounts, groups, and servers as `MDI Sensitive Entities – Tier0 – Prod` (Domain/Enterprise Admins, DCs, AD FS/AD CS, PKI, backup, break-glass).

3. Preview the change first with `MDIFULL-Deploy.ps1 -Mode SensitiveTag -InputCsv .\tier0.csv` (safe preview by default); apply only with `-Apply`.

4. Run `MDIFULL-Validate.ps1 -Check Sensitive` to confirm the entities are tagged.

Step 5 — Plant and tag honeytokens

1. In AD, create the idle decoy account(s) `MDI Honeytoken – Decoy Identity – Pilot` with believable names; confirm **nothing legitimate uses them** (no logon scripts, services, or scheduled tasks).

2. `security.microsoft.com` → **Settings** → **Identities** → **Entity tags** → **Honeytoken** — add the decoy account(s).

3. Preview with `MDIFULL-Deploy.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv` (safe preview); apply with `-Apply`.

4. Run `MDIFULL-Validate.ps1 -Check Honeytoken` to confirm the accounts are tagged as honeytokens.

Step 6 — Enable and triage identity posture (ISPM)

1. After the pilot sensor is healthy, `security.microsoft.com` → **Secure Score** → **Recommended actions** (or **Exposure management** → **Recommendations**), filter to **Identity** — MDI ISPM recommendations appear (unconstrained delegation, spooler on DCs, legacy protocols, risky certificate templates). *Verify current ISPM availability; allow warm-up.*

2. Record the starting posture with `MDIFULL-Report.ps1 -SaveBaseline` and build the `MDI ISPM Shortlist – Tier0 – Prod` (top 3–5 by impact and effort); assign owners.

3. *Which posture items are "must-fix" and the remediation change-approval are the security/AD team's decisions — this guide surfaces the recommendations and the technical fix path, it does not enforce a

change.*

Step 7 — Validate detections on the pilot

1. Run `MDIFULL-Validate.ps1 -Check Signal` to execute `MDI Detection Validation – Tenant – Validate` — it confirms the identity tables (`IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `IdentityQueryEvents`) are streaming from the pilot sensor.

2. Trigger a **benign, controlled** detection: authenticate against or query the honeytoken `MDI Honeytoken – Decoy Identity – Pilot`, or run a Microsoft-documented safe simulation. Confirm an alert appears in `security.microsoft.com` → **Incidents & alerts**.

3. Confirm response actions remain **approval-gated** (nothing auto-acted). Complete the Phase-A tests in the Testing Guide before enabling the team.

Phase B — Enable the team (five hands-on labs)

Run the labs in order against the **pilot** scope. All steps are **read-only or preview-first**; the one configuration action (already done in Step 5 — the honeytoken tag) and the response rehearsal are **facilitator-driven and approval-gated**. Every attendee performs the steps.

Lab 0 — Readiness check

1. Facilitator signs in as **Security Administrator**; confirm the pilot sensor is **Healthy** (Step 3).

2. Run `MDIFULL-Discovery.ps1` and `MDIFULL-Validate.ps1` — confirm a healthy sensor, ISPM data present, hunting access, and read-scope consent. **Pass gate:** both green.

Lab 1 — Identity attack paths & the entity page

1. Open **Assets** → **Identities** (or search a username). Open a **user** entity page for a pilot account and walk the panels: **Overview** (investigation-priority score, tags, risk), **Timeline** (authentications, alerts, directory changes), **Lateral movement paths** (how this identity could reach higher-privilege accounts).

2. Open a **device** (domain controller) entity page; note logons observed and related identities.

3. **Pass gate:** each attendee, on a second entity page, correctly points to the timeline, the lateral-movement path panel, and the investigation-priority score and explains each.

Lab 2 — Recon & lateral-movement detections

1. Open **Incidents & alerts** → **Alerts**; filter **Service source = Defender for Identity**. Review the four detection families (recon, compromised credentials, lateral movement, domain dominance) and map each to its MITRE tactic.

2. Open **Hunting** → **Advanced hunting**. Paste the **Recon** and **Lateral movement** sections of `MDIFULL-LabQueries.kql`, run them, and **Save as** `Recon – Identity – Hunt` and `LateralMovement – Identity – Hunt`.

3. Take one alert (or query hit) through triage: what identity, what technique, is it expected, what would escalate it.

4. **Pass gate:** each attendee maps a chosen detection to its MITRE tactic and states the first two triage questions.

Lab 3 — Honeytokens

1. Confirm the decoy `MDI Honeytoken – Decoy Identity – Pilot` is tagged (from Step 5) and has **no legitimate sign-ins**.

2. Explain the model: a honeytoken should *never* authenticate, so **any** logon/query is high-fidelity; MDI raises **"Honeytoken activity"** when it is touched.

3. In **Advanced hunting**, paste the **Honeytoken** section of `MDIFULL-LabQueries.kql`, run it, and **Save as** `Honeytoken – Identity – Hunt`.

4. *(Optional, lab environment only)* From a test workstation, attempt a benign authentication as the honeytoken to generate the detection, then observe the alert. **Do not do this in production.**

5. **Pass gate:** the saved query returns the account and attendees can explain why any activity is a real signal.

Lab 4 — ISPM posture (read and prioritize, no enforcement)

1. Open **Secure Score** (or **Exposure management** → **Recommendations**) and filter to **Identity** / **Defender for Identity**. Walk the common recommendations (remove unconstrained delegation, disable legacy/weak protocols, address exposed credentials, review dormant privileged accounts, SAM-R / spooler hardening) and note impact and affected entities.

2. Run `MDIFULL-Report.ps1` to export a read-only posture summary; refine the `MDI ISPM Shortlist – Tier0 – Prod` (top 3–5) with owner candidates.

3. **Boundary:** the labs **do not enforce** any change — remediation and change approval belong to the customer's security/ops functions.

4. **Pass gate:** a written shortlist exists and each attendee can explain why the top item ranks first.

Lab 5 — Investigation end-to-end + response rehearsal

1. Pick a lead: a real alert from Lab 2, the honeytoken hit from Lab 3, or an entity from Lab 1.

2. Build the story on the **user entity page**: timeline → related alerts → lateral-movement path → involved devices. Cross-check in **Advanced hunting** with the saved queries (or the **Investigation** section of `MDIFULL-LabQueries.kql`, saved as `Investigation – Identity – Hunt`).

3. Capture findings in `MDI Investigation – Identity – Validate`: what happened, which identities/devices, which technique, confidence, recommended action.

4. **Response rehearsal (approval-gated):** from the entity page, review **Disable user / Force password reset / Suspend** and discuss when each applies and who must approve. **Describe/queue the action for approval; do not execute against a production identity.** Discuss how XDR **automatic attack disruption** changes the playbook.

5. **Pass gate:** a completed investigation doc plus each attendee correctly stating which response action fits and who approves it. Run the knowledge-check in the Testing Guide.

Phase C — Prove the value (guided PoC)

Run the PoC on the same pilot DC with **benign, simulated** signal only. All test signal is harmless read-only enumeration and a honeytoken touch — **never a real attack, exploit, or credential theft**.

Step 8 — Generate benign test signal

1. From the **test workstation**, signed in as the **test account**, run `MDIFULL-TestSignal.ps1 -HoneytokenSam "<sam>"` (preview first with `-WhatIf`). It performs only **benign, read-only** actions tagged `MDI-TESTSIGNAL`:

• **Reconnaissance signal:** harmless enumeration — `net group "Domain Admins" /domain`, `net user /domain`, `nltest /domain_trusts`, and a read-only LDAP query. MDI flags the *pattern* as reconnaissance.

• **Honeytoken signal:** a benign lookup touching `MDI Honeytoken – Decoy Identity – Pilot`.

1. **Do not** run attack tooling, credential-dumping utilities, exploitation frameworks, or anything that changes AD.

2. Note the time of each signal for the scorecard latency comparison.

Step 9 — Validate, demonstrate, and score

1. Run `MDIFULL-Validate.ps1 -Check Signal` — confirm (read-only) the **reconnaissance** and **honeytoken** alerts reached the portal (`AlertInfo`), the activity appears in the identity tables, and the identity timeline populated.

2. In the portal, walk the demo for stakeholders: incident → **alert story** → **identity page** → **Timeline** → **UEBA / investigation-priority** view.

3. Run `MDIFULL-Report.ps1 -Scorecard -CriteriaPath .\criteria.json -OutFile "MDI PoC Scorecard – Pilot DC – Validate.json"` — it maps each observation to your pass/fail criteria and writes the scorecard. Review it with stakeholders as the decision artefact.

Step 10 — Tear down the PoC-only scope

1. Run `MDIFULL-Deploy.ps1 -Rollback -WhatIf` first to preview, then for real.

2. Untag any **PoC-only** decoy created just for the demo, **resolve** the `MDI-TESTSIGNAL` test alerts in **Incidents & alerts** as "Not a threat / test," and confirm no PoC-only DSA/sensor residue remains with `MDIFULL-Validate.ps1`.

3. **Keep** the production sensor, the retained honeytoken(s), the tags, the saved queries, the ISPM shortlist, and the scorecard. Only the *demo-only* artefacts are removed.

Expand (across the fleet)

Step 11 — Expand to remaining DCs and AD FS/AD CS

1. **Sensors:** install the sensor on each remaining domain controller in change windows until all show **Running / Healthy**; then AD FS / AD CS servers.
2. **Tags/honeytokens:** extend sensitive tagging and add honeytokens across additional forests/domains as needed.
3. **ISPM:** re-run `MDIFULL-Report.ps1 -SaveBaseline` after full sensor coverage — recommendations get more complete as more DCs report.
4. **Team:** use the saved queries, runbook, and investigation doc to onboard new analysts.
5. **Response:** only after coverage and detections are trusted, review response-action automation with the SOC (still their decision).
6. Track sensor-health and posture drift each cycle with `MDIFULL-Report.ps1`.

Order of operations

`Assess → activate MDI + DSA → install pilot-DC sensor → tag sensitive entities → plant honeytokens → enable/triage ISPM → validate detections → run the five enablement labs → generate benign PoC signal → validate + demonstrate + score → tear down PoC-only scope → expand to all DCs + AD FS/AD CS (response approval-gated throughout)`

5. Testing Guide — Microsoft Defender for Identity

Validate each phase before moving to the next. Every test has a **pass/fail gate**; do not expand to the remaining domain controllers until the pilot passes all phases. Objects are referenced by the exact names from the Implementation Guide.

Part A — Deploy validation (Phase A)

A1 — DSA configured

Procedure: Run `MDIFULL-Validate.ps1 -Check DSA`. In **Settings** → **Identities** → **Directory Service accounts**, confirm `MDI Directory Service Account – Tenant – Prod` is present. **Pass:** The DSA (gMSA) is registered and the sensor can retrieve its password. **Fail:** DSA missing or password-retrieval fails — see Troubleshooting *DSA not working*.

A2 — Pilot sensor healthy

Procedure: Run `MDIFULL-Validate.ps1 -Check Sensor`. In **Settings** → **Identities** → **Sensors**, confirm `MDI Sensor Host – Pilot DC – Pilot` is **Running / Healthy**. **Pass:** The pilot sensor is healthy and reporting; no health alerts against it. **Fail:** Sensor "Unhealthy / Not configured / Stopped" — see Troubleshooting *Sensor unhealthy*.

A3 — Sensitive entities tagged

Procedure: Run `MDIFULL-Validate.ps1 -Check Sensitive`. In **Entity tags** → **Sensitive**, confirm the `MDI Sensitive Entities – Tier0 – Prod` accounts/groups/servers are tagged. **Pass:** All planned Tier-0 entities are tagged sensitive. **Fail:** Tier-0 entities missing — re-apply per Implementation Step 4.

A4 — Honeytokens tagged

Procedure: Run `MDIFULL-Validate.ps1 -Check Honeytoken`. In **Entity tags** → **Honeytoken**, confirm `MDI Honeytoken – Decoy Identity – Pilot` is tagged. **Pass:** The decoy account(s) are tagged and remain unused. **Fail:** Not tagged, or the account shows real usage — re-apply per Implementation Step 5 and confirm nothing legitimate uses the account.

A5 — Identity signal streaming

Procedure: Run `MDIFULL-Validate.ps1 -Check Signal` to execute `MDI Detection Validation – Tenant – Validate`. Confirm `IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, and `IdentityQueryEvents` return recent rows. **Pass:** All four identity tables stream from the pilot sensor. **Fail:** A table is empty — see Troubleshooting *No identity signal*.

A6 — A detection actually fires

Procedure: Trigger a benign, controlled detection (honeytoken touch or Microsoft-documented safe simulation). In **Incidents & alerts**, confirm the resulting alert appears. **Pass:** The controlled test produces a visible MDI alert within the expected window. **Fail:** No alert — see Troubleshooting *Detection does not fire*.

A7 — ISPM recommendations present

Procedure: Run `MDIFULL-Report.ps1 -SaveBaseline`. In **Secure Score** → **Recommended actions** (Identity), confirm MDI ISPM recommendations are listed and the `MDI ISPM Shortlist – Tier0 – Prod` is captured. **Pass:** ISPM recommendations appear and a posture baseline is recorded. **Fail:** No recommendations — allow warm-up, confirm sensor health (A2), re-run; see Troubleshooting *No ISPM recommendations*.

A8 — Response actions approval-gated

Procedure: Run `MDIFULL-Report.ps1` and confirm response actions are approval-gated (not automatic) per Implementation Step 7. **Pass:** No response action auto-executes; actions require approval. **Fail:** Automation is higher than agreed — reset per Implementation Step 7 and confirm with the SOC.

Phase A exit gate: A1–A8 pass on the pilot DC before enabling the team.

Part B — Team enablement (Phase B knowledge-check)

Score each attendee; passing = correct on at least **8 of 10**.

1. **Sensors:** Which servers host MDI sensors and what does each observe? *(DCs, AD CS, AD FS/Entra Connect — authentication and directory traffic/behavior.)*

2. **Attack path:** On an entity page, where are lateral-movement paths and the investigation-priority score, and what do they mean?

3. **Detection families:** Name the four families with one detection each *(recon, compromised credentials, lateral movement, domain dominance)*.

4. **MITRE:** Map Pass-the-Ticket and DCSync to their MITRE tactics *(Lateral Movement; Credential Access)*.

5. **Honeytoken:** What makes a good honeytoken and why is any activity on it high-fidelity?

6. **Hunting:** Which advanced-hunting tables hold identity telemetry? *(IdentityLogonEvents, IdentityQueryEvents, IdentityDirectoryEvents, IdentityInfo.)*

7. **ISPM:** Give two recommendations and the risk each addresses.

8. **Investigation:** Outline the steps to investigate a suspicious identity from entity page to advanced hunting.

9. **Response & governance:** What are the MDI response actions and **who** must approve them?

10. **Boundary:** Which decisions in an identity incident belong to the SOC/security team rather than the tool operator?

Capstone (per attendee, unaided, on a second entity/alert): open the entity page and read the timeline + lateral-movement path; run one saved hunting query and interpret a result; state the recommended response action and the required approval.

Phase B exit gate: every attendee meets ≥8/10 and completes the capstone; the take-home artefacts exist (the saved queries `Recon/LateralMovement/Honeytoken/Investigation – Identity – Hunt`, the honeytoken tag, the `MDI ISPM Shortlist – Tier0 – Prod`, and the `MDI Investigation – Identity – Validate` doc).

Part C — PoC validation (Phase C)

C1 — Benign recon signal generated

Procedure: From the test workstation/account, run `MDIFULL-TestSignal.ps1`; confirm the enumeration commands and the honeytoken touch completed. **Pass:** The benign read-only enumeration ran and the honeytoken was touched. No attack tooling used. **Fail:** Commands blocked — confirm the test account can query AD; see Troubleshooting *Recon signal not generated*.

C2 — Reconnaissance alert reached the portal

Procedure: Run `MDIFULL-Validate.ps1 -Check Signal`. In **Incidents & alerts**, confirm a reconnaissance-type alert from the pilot activity. **Pass:** A real reconnaissance alert appears within your agreed latency window. **Fail:** No alert — see Troubleshooting *No reconnaissance alert*.

C3 — Honeytoken activity flagged

Procedure: Confirm the honeytoken touch raised an alert / is highlighted; `MDIFULL-Validate.ps1 -Check Signal` checks `AlertInfo` and identity events. **Pass:** Activity against `MDI Honeytoken – Decoy Identity – Pilot` is flagged as high-fidelity. **Fail:** Not flagged — see Troubleshooting *Honeytoken not flagged*.

C4 — Identity timeline & hunting visibility

Procedure: Open the identity page → **Timeline**; `MDIFULL-Validate.ps1 -Check Signal` runs KQL over the identity tables. **Pass:** The timeline shows the test activity and hunting tables return recent rows.

Fail: Timeline/tables empty — allow warm-up; see Troubleshooting *No identity signal*.

C5 — Scorecard produced

Procedure: Run `MDIFULL-Report.ps1 -Scorecard` and confirm `MDI PoC Scorecard – Pilot DC – Validate.json` is written with a pass/fail per criterion. **Pass:** A complete scorecard maps every observation to your criteria. **Fail:** Missing observations — re-run the relevant validate step first.

C6 — Clean PoC teardown

Procedure: Run `MDIFULL-Deploy.ps1 -Rollback -WhatIf`, then for real; re-run `MDIFULL-Validate.ps1`. **Pass:** PoC-only decoys untagged, `MDI-TESTSIGNAL` alerts resolved, no PoC-only residue; production sensor and retained artefacts intact. **Fail:** Residue remains — see Troubleshooting *Teardown left residue*.

Phase C exit gate: C1–C5 pass to call the PoC conclusive; C6 passes to close it out cleanly. Keep the scorecard file as the decision record.

Overall exit gate

All Phase A tests, the Phase B knowledge-check, and Phase C tests pass on the pilot **before** expanding to the remaining domain controllers (Implementation Step 11). Re-run A2, A5, A6, and A7 after each sensor-expansion wave and compare against the saved posture baseline.

6. Troubleshooting Guide — Microsoft Defender for Identity

Common field issues across all three phases, in **symptom** → **cause** → **fix** form. Run `MDIFULL-Discovery.ps1` and `MDIFULL-Validate.ps1` first — most causes surface there. All PoC test signal is benign (read-only enumeration + a honeytoken decoy). For category-level prerequisites (licensing, roles, network) see `global-requirements-defender.md`.

DSA not working

Symptom: Test A1 fails — `MDI Directory Service Account – Tenant – Prod` is missing, or the sensor cannot use it and enrichment detections are limited. **Cause:** The gMSA is not registered under **Directory Service accounts**, the sensor host computer accounts lack permission to retrieve the gMSA password, or a standard account's password is wrong. **Fix:** Confirm the DSA is registered under **Settings** → **Identities** → **Directory Service accounts**; grant the sensor hosts permission to retrieve the gMSA password; re-enter credentials for a standard account. *Verify current MDI DSA requirements.* Re-run `MDIFULL-Validate.ps1 -Check DSA`.

Sensor unhealthy

Symptom: `MDI Sensor Host – Pilot DC – Pilot` shows **Unhealthy / Not configured / Stopped** in **Settings** → **Identities** → **Sensors**. **Cause:** Blocked outbound reach to the MDI service, insufficient DC resources, an unsupported sensor OS, a mis-scoped DSA, a wrong/expired access key, or a proxy in the way. **Fix:** Confirm the DC can reach the MDI service URLs (per `global-requirements-defender.md`); confirm DC resources and a supported OS version (*verify current sensor OS support*); verify the DSA; reinstall with a fresh access key from **Add sensor** if needed;

configure the sensor proxy if one is required. Re-run `MDIFULL-Validate.ps1 -Check Sensor`.

No identity signal

Symptom: Test A5 / C4 fails — one or more of `IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `IdentityQueryEvents` return no rows; entity-page timelines are empty. **Cause:** The sensor is unhealthy or too newly installed, not enough traffic has passed the DC yet, or the account lacks hunting permission. **Fix:** Confirm the pilot sensor is healthy first (A2). Allow warm-up — identity tables populate over minutes to hours. Confirm the account holds advanced-hunting read (Requirement 2.8). Generate benign authentication traffic through the DC and re-run `MDIFULL-Validate.ps1 -Check Signal`.

Detection does not fire

Symptom: Test A6 fails — the controlled honeytoken touch or safe simulation produces no alert. **Cause:** The honeytoken is not actually tagged, the sensor is unhealthy, the test did not reach a sensored DC, or not enough time has passed. **Fix:** Confirm the honeytoken is tagged (A4) and the sensor healthy (A2); ensure the authentication/query targets a DC that has a sensor; allow time for the alert to surface in **Incidents & alerts**. Re-test with `MDIFULL-Validate.ps1 -Check Signal` to confirm the query event is recorded.

No ISPM recommendations

Symptom: Test A7 / Lab 4 fails — no MDI Identity recommendations appear in **Secure Score** → **Recommended actions** or **Exposure management** → **Recommendations**. **Cause:** Sensors are too newly onboarded for ISPM to compute, sensor coverage is too thin, ISPM is not yet available in the tenant, or you are looking at the wrong blade. **Fix:** Confirm sensor health (A2) and allow warm-up; expand sensor coverage so more DCs report; try both **Secure Score** and **Exposure management** → **Recommendations** (menus move — *verify current navigation*); *verify current ISPM availability in your tenant.* Re-run `MDIFULL-Report.ps1 -SaveBaseline`.

Honeytoken generating noise

Symptom: The honeytoken account fires alerts from legitimate activity. **Cause:** A real process (logon script, service, scheduled task, or a person) is using the decoy account, so it is no longer idle. **Fix:** Identify and remove the legitimate use (the **Honeytoken** section of `MDIFULL-LabQueries.kql` shows what touched it), or retire the account and create a genuinely idle replacement named `MDI Honeytoken – Decoy Identity – Pilot`. Re-tag and re-validate (A4).

Honeytoken candidate list empty (`signInActivity` blank)

Symptom: `MDIFULL-Discovery.ps1` lists no candidate accounts or blank last-sign-in. **Cause:** `signInActivity` requires **Entra ID P1/P2** and `AuditLog.Read.All`; without them the property is empty. **Fix:** Confirm the Entra plan and the scope (*verify availability*). If unavailable, the facilitator picks a decoy account manually (create `svc-legacy-backup` in the lab) — the honeytoken does not depend on this list.

Entity pages / advanced hunting empty during the labs

Symptom: `IdentityInfo | take 5` returns nothing; user entity pages have empty timelines. **Cause:** Sensors only just onboarded (warm-up), the `Identity*` tables are not yet populated, or the attendee lacks hunting permission. **Fix:** Allow a warm-up window; confirm the attendee role includes advanced-hunting read (Requirement 2.8, 2.11). A quiet tenant is fine for teaching — use the runbook descriptions and the honeytoken lab to generate signal in a **lab**. Re-run `MDIFULL-Validate.ps1`.

Response actions greyed out (Lab 5)

Symptom: Disable user / Force password reset are unavailable on the entity page. **Cause:** The attendee holds a read-only role (by design), or the action requires higher privilege / the approval gate. **Fix:** This is expected — attendees learn read-only and **rehearse** response. Only the facilitator (with the appropriate role) demonstrates the action, and only in a lab / with approval. Do not execute against a production identity during training.

Recon signal not generated (PoC)

Symptom: Test C1 fails — the enumeration commands are blocked or return nothing. **Cause:** The test account cannot query AD, or a security tool blocks the enumeration on the workstation. **Fix:** Confirm the test account can run `net group` / `nltest` / LDAP queries against the domain; run `MDIFULL-TestSignal.ps1` from a domain-joined test workstation.

No reconnaissance alert (PoC)

Symptom: Test C2 fails — no reconnaissance alert after the benign enumeration. **Cause:** The sensor is not healthy, the enumeration was too light to trip the detection threshold, or signal has not propagated. **Fix:** Confirm sensor health (A2); run the full enumeration set (the script runs several patterns); allow propagation time; re-run `MDIFULL-Validate.ps1 -Check Signal`. Some detections need a recognisable enumeration pattern, not a single query.

Teardown left residue (PoC)

Symptom: After `MDIFULL-Deploy.ps1 -Rollback`, a PoC-only decoy is still tagged or `MDI-TESTSIGNAL` alerts remain open. **Cause:** Rollback ran in `-WhatIf` only, or the alerts were not resolved. **Fix:** Run rollback **without** `-WhatIf`; untag any PoC-only decoy under **Entity tags** → **Honeytoken**; resolve the `MDI-TESTSIGNAL` alerts in **Incidents & alerts** as "Not a threat / test." Re-confirm with `MDIFULL-Validate.ps1`. Note: the production sensor, retained honeytokens, tags, and saved queries are **kept** — only demo-only artefacts are removed.

Response action acted without approval

Symptom: A user was disabled or a password reset forced automatically. **Cause:** A response action was configured for automatic execution rather than approval-gated. **Fix:** Reset the action to approval-gated (Implementation Step 7), confirm with the SOC, and record the configuration with `MDIFULL-Report.ps1`. *The automation-guardrail decision is the security team's.*

Script cannot authenticate

Symptom: `MDIFULL-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, consent to the required read scopes was not granted, the account lacks Security Reader, or the AD read helpers are run off a machine without the AD module. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm Security Reader (or Security Administrator for the honeytoken tag); run the AD-inventory portion from a domain-joined host with `RSAT-AD-PowerShell`. Re-run the script.

Portal menu paths don't match

Symptom: A navigation path in the guide doesn't exist. **Cause:** Microsoft periodically relocates Defender portal menus. **Fix:** Use global search (top bar) for the feature name (e.g. "Honeytoken", "Entity tags", "ISPM", "Sensors"). *Verify current navigation in your tenant* and update your copy of the guide.

7. Best Practices — Microsoft Defender for Identity

Field-proven guidance for keeping the Defender for Identity deployment healthy, high-signal, low-risk — and for enabling the team and proving the value well.

Deploy in a ring, never all at once

• Install the sensor on **one pilot domain controller** (`MDI Sensor Host – Pilot DC – Pilot`) first, health-check it, validate a detection, then expand DC-by-DC in change windows. A bad rollout on every DC simultaneously is avoidable.

• Cover **every** domain controller eventually — a single un-sensored DC is a blind spot attackers can use. Extend sensors to **AD FS and AD CS** servers; they are prime attacker targets and MDI has detections specific to them.

Protect and monitor the sensors

• Domain-controller sensors are security-critical infrastructure. Keep them healthy, patched, and monitored; a silent sensor blinds MDI. Review sensor health on a cadence with `MDIFULL-Report.ps1`.

Use a gMSA for the Directory Service Account

• Prefer a **group Managed Service Account** for `MDI Directory Service Account – Tenant – Prod` so there is no static password to rotate or leak. Grant only the read access MDI needs. *Verify current MDI DSA requirements.*

Honeytokens are your highest-signal detection

• Plant decoy accounts (`MDI Honeytoken – Decoy Identity – Pilot`) that **nothing legitimate ever touches**, tag them, and treat any hit as high-confidence. Give them believable names and place them where an attacker enumerating AD would find them. Keep them genuinely idle — a honeytoken used by a real process becomes noise. **Never test a honeytoken in production** — only in a lab.

Tag Tier-0 so alerts are weighted correctly

• Tag Domain/Enterprise Admins, DCs, AD FS/AD CS, PKI, backup, and break-glass as sensitive (`MDI Sensitive Entities – Tier0 – Prod`). MDI weights alerts on tagged entities; untagged Tier-0 dilutes the signal that matters most.

Work ISPM as a trend, not a one-off

• Triage the **identity posture** recommendations by exploitability (unconstrained delegation, spooler on DCs, legacy protocols, risky certificate templates) and track the delta each cycle with `MDIFULL-Report.ps1 -SaveBaseline`. Re-baseline after full sensor coverage. Keep a short shortlist (`MDI ISPM Shortlist – Tier0 – Prod`) rather than trying to fix everything at once.

Enable the team on a pilot, read-only by default

• Teach against a **pilot/lab, not production privileged identities**. The only configuration action in the labs is tagging the honeytoken (facilitator); everything else is discovery, hunting, and reading. Use consistent object names so the take-home artefacts are findable, and make sure every attendee leaves with the saved queries, the honeytoken tag, the ISPM shortlist, and the investigation doc.

• Instill the habits: **work the entity page first** (investigation priority + timeline + lateral-movement path answers most triage before advanced hunting); **treat honeytoken activity as real until disproven**; **hunt with saved queries** to make investigation repeatable; **respect the approval gate** — a clean, evidenced hand-off, not a unilateral action.

Prove value with benign signal only — never real attacks

• Use **read-only directory enumeration** and a **honeytoken** touch exclusively for the PoC. MDI detects the *pattern* of enumeration from benign commands — you do not need a real attack to prove it. **Never** run credential-dumping tools, exploitation frameworks, DCSync, or anything that modifies AD; adversary emulation belongs in a separate, governed engagement.

• Set the **evaluation criteria in writing first** and feed them to `MDIFULL-Report.ps1 -Scorecard` so the verdict is objective. Allow **warm-up** (timeline, hunting tables, and UEBA populate over minutes to hours) before scoring a "fail." Tear down the PoC-only scope completely and keep only the scorecard.

Keep response actions approval-gated until trusted

• Leave MDI response actions (disable user, force password reset) approval-gated during the pilot and early expansion. Moving toward automatic response is the SOC/security team's decision, made only after coverage and detections are proven. MDI also feeds XDR **automatic attack disruption** — factor that into the playbook.

Keep licensing and preview facts current

• MDI and some features are **license-** or **preview-gated** and evolve, and sensor OS support and endpoint URLs change. Before relying on a capability, **verify current availability, sensor support, and the endpoint list in your tenant** — do not assume a feature is included.

Common anti-patterns to avoid

- Standing up one sensor and calling MDI "deployed."
- Using an active account as a honeytoken (false positives destroy trust in the signal).
- Enforcing ISPM changes mid-enablement without change approval.
- Executing a response action against a production identity "to see what happens."
- Running attack tooling in the PoC instead of benign signal.

Where to go next

Once MDI is deployed, the team is enabled, and the value is proven, natural next steps in the catalog:

- **Microsoft Defender XDR — Unified Onboarding** to bring MDI together with Endpoint, Office 365, and Cloud Apps into one correlated portal.
- **SOC Operations — Sentinel + Defender XDR** to stream MDI into a SIEM for long-term hunting and SOC process design.
- **Advanced Threat Hunting (KQL)** to go deeper on cross-pillar hunting beyond the identity tables.
- **Entra ID governance / Conditional Access / PIM** and **Active Directory hardening / tiered admin** solutions to harden the identity layer end to end alongside on-prem AD.

8. Deliverables Summary — Microsoft Defender for Identity

Documentation (source/)

Document	Purpose
Executive Summary	Business case, the end-to-end outcome, the boundary, and effort
1. Solution Overview	What MDI is — sensors, DSA, honeytokens, sensitive tags, ISPM, detections, the operator console, and how a safe PoC proves it; the three phases; benefits, limitations
2. Requirements Guide	Solution-specific prerequisites (deploy, enable, prove) with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	One continuous playbook — Phase A Deploy (DSA, pilot sensor, tags, honeytokens, ISPM, validate), Phase B Enable (five labs), Phase C Prove (guided PoC), then Expand
5. Testing Guide	Phase-A deploy tests, the Phase-B knowledge-check + capstone, and Phase-C PoC tests, each with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation across all three phases

Document	Purpose
7. Best Practices	Field-proven recommendations and where to go next

Automation assets (scripts/)

Asset	What it does					
`MDIFULL-Discovery.ps1`	Read-only readiness (Microsoft Graph Security + AD helpers) — MDI activation/workspace, plan hints, admin roles, every domain controller and OS, DSA readiness, existing sensor health, and candidate no-activity honeytoken accounts. Makes no changes					
`MDIFULL-Deploy.ps1`	`-WhatIf`-gated change helper — tags sensitive entities (`-Mode SensitiveTag`) and honeytokens (`-Mode Honeytoken`); `-Rollback` guides the clean teardown of the PoC-only scope. Safe preview by default; applies only with `-Apply`					

Asset	What it does					
`MDIFULL-Validate.ps1`	Read-only validation (`-Check DSA\	Senso r\	Sensitiv e\	Honeytoke n\	Signa l\	All) — confirms the DSA, sensor health, sensitive/honeytoken tags, and (via MDI Detection Validation – Tenant – Validate`) that the identity tables stream and a test detection is visible
`MDIFULL-TestSignal.ps1`	**Benign** PoC test-signal generator — runs harmless read-only directory enumeration and touches the honeytoken (no attack tooling, ever); `-WhatIf` previews					
`MDIFULL-Report.ps1`	Read-only posture/coverage report — sensor health, Identity Secure Score / ISPM recommendations, baseline drift (`-SaveBaseline`), response-action posture; `-Scorecard` maps PoC observations to your criteria and writes `MDI PoC Scorecard – Pilot DC – Validate`					

Asset	What it does					
`MDIFULL-LabQueries.kql`	Advanced-hunting KQL pack for the labs and detection validation — recon, lateral movement, honeytoken activity, suspicious-auth volume, single-account investigation, and identity-context queries over the `Identity*` tables					

Take-home artefacts (produced during the engagement)

- Saved hunting queries: `Recon – Identity – Hunt`, `LateralMovement – Identity – Hunt`, `Honeytoken – Identity – Hunt`, `Investigation – Identity – Hunt`, and `MDI Detection Validation – Tenant – Validate`.
- The honeytoken tag `MDI Honeytoken – Decoy Identity – Pilot` and the sensitive-entity tag `MDI Sensitive Entities – Tier0 – Prod`.
- The `MDI ISPM Shortlist – Tier0 – Prod` remediation shortlist.
- The `MDI Investigation – Identity – Validate` findings document.
- The `MDI PoC Scorecard – Pilot DC – Validate` decision record.

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All six **automation assets** (five PowerShell scripts with embedded KQL, plus the standalone KQL pack) in a single ZIP.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — deploy the pilot sensor, enable the team, prove the value, and expand, with response actions kept approval-gated until trusted.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A fully sensored, team-ready, value-proven Defender for Identity delivering ITDR — sensors healthy on every domain controller, the DSA configured, honeytokens planted and Tier-0 tagged, ISPM triaged, detections proven to fire, a team that can investigate unaided, and a scorecard the business can stand

behind — the whole lifecycle in one package.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MDIFULL-Deploy.ps1

Purpose: The only change helper in the "Microsoft Defender for Identity" solution. Two modes: -Mode SensitiveTag | Honeytoken : add the planned entities to the MDI Sensitive tag ('MDI Sensitive Entities - Tier0 - Prod') or the Honeytoken tag ('MDI Honeytoken - Decoy Identity - Pilot'). Defaults to a safe preview; applies tags only when explicitly confirmed with -Apply. -Rollback : guide the clean teardown of the PoC-only scope — untag any demo-only decoy, resolve the 'MDI-TESTSIGNAL' test alerts, and confirm no PoC-only residue. The production sensor, retained honeytokens, tags, and saved queries are KEPT. HOW TO RUN Preview a sensitive/honeytoken tagging (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Deploy.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Deploy.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv -Apply Guided PoC rollback (preview then real): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Deploy.ps1 -Rollback -WhatIf The CSV has one column: samAccountName (or the entity object id). Requires Microsoft.Graph with the MDI tag write scope for -Apply. EXPECTED RESULTS Preview: prints each resolved entity and the tag it WOULD receive. Apply: applies the tag and confirms. Rollback: prints the exact untag / resolve steps. .NOTES White-label. DEFAULTS TO preview. Honeytokens must be genuinely idle accounts. Rollback removes only demo-only artefacts — never the production sensor or retained honeytokens. > [CmdletBinding(SupportsShouldProcess = $true)] param([ValidateSet('SensitiveTag','Honeytoken')] [string]$Mode, [string]$InputCsv, [switch]$Apply, [switch]$Rollback, [string]$Tag = 'MDI-TESTSIGNAL')

How to run: Preview a sensitive/honeytoken tagging (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Deploy.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv Apply (explicit — required to make the change): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Deploy.ps1 -Mode Honeytoken -InputCsv .\honeytokens.csv -Apply Guided PoC rollback (preview then real): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Deploy.ps1 -Rollback -WhatIf The CSV has one column: samAccountName (or the entity object id). Requires Microsoft.Graph with the MDI tag write scope for -Apply. EXPECTED RESULTS Preview: prints each resolved entity and the tag it WOULD receive. Apply: applies the tag and confirms. Rollback: prints the exact untag / resolve steps. .NOTES White-label. DEFAULTS TO preview. Honeytokens must be genuinely idle accounts. Rollback removes only demo-only artefacts — never the production sensor or retained honeytokens. > [CmdletBinding(SupportsShouldProcess = $true)] param([ValidateSet('SensitiveTag','Honeytoken')] [string]$Mode, [string]$InputCsv, [switch]$Apply, [switch]$Rollback, [string]$Tag = 'MDI-TESTSIGNAL')

MDIFULL-Discovery.ps1

Purpose: Read-only discovery for the "Microsoft Defender for Identity" solution. Reports the signed-in context, licensed Defender-for-Identity plan hints, the admin roles held, every domain controller (and AD FS/AD CS candidate) with its OS, Directory Service Account readiness, existing sensor health, and candidate no-activity accounts you could designate as honeytokens. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDIFULL-Discovery.ps1 Requires the Microsoft.Graph modules (Microsoft.Graph.Authentication, Microsoft.Graph.Identity.DirectoryManagement). The AD inventory portion uses the RSAT ActiveDirectory module and should run from a domain-joined host. Connects read-only with Connect-MgGraph and prompts for consent to read scopes. EXPECTED

RESULTS A readiness picture — tenant/context, plan hints that indicate MDI entitlement, admin roles, the domain-controller inventory to sensor, DSA notes, sensor health, and honeytoken candidates — the state to confirm before you begin. Written to a JSON snapshot. .NOTES White-label. Read-only. Reports on configuration/identifiers only; makes no changes. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDIFULL-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\MDIFULL-Discovery.ps1 Requires the Microsoft.Graph modules (Microsoft.Graph.Authentication, Microsoft.Graph.Identity.DirectoryManagement). The AD inventory portion uses the RSAT ActiveDirectory module and should run from a domain-joined host. Connects read-only with Connect-MgGraph and prompts for consent to read scopes. EXPECTED RESULTS A readiness picture — tenant/context, plan hints that indicate MDI entitlement, admin roles, the domain-controller inventory to sensor, DSA notes, sensor health, and honeytoken candidates — the state to confirm before you begin. Written to a JSON snapshot. .NOTES White-label. Read-only. Reports on configuration/identifiers only; makes no changes. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDIFULL-discovery.json")

MDIFULL-LabQueries.kql

MDIFULL-Report.ps1

Purpose: Read-only reporting for the "Microsoft Defender for Identity" solution. Default mode summarizes sensor health across domain controllers, the MDI/Identity Secure Score recommendations (ISPM), and — with -SaveBaseline — records a posture baseline so later runs show drift; it also reports the response-action posture (should be approval-gated). -Scorecard re-checks the key MDI capabilities (recon/honeytoken alert, enumeration observed, directory activity, logon context) via read-only advanced hunting, maps each to the pass/fail criteria you agreed up front, and writes a JSON + console scorecard. HOW TO RUN Posture / coverage report (create or compare a baseline): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Report.ps1 -SaveBaseline powershell -ExecutionPolicy Bypass -File .\MDIFULL-Report.ps1 PoC scorecard against your criteria: powershell -ExecutionPolicy Bypass -File .\MDIFULL-Report.ps1 -Scorecard ` -CriteriaPath .\criteria.json -OutFile "MDI PoC Scorecard - Pilot DC - Validate.json" Sample criteria.json: [{ "id":"alert", "label":"Recon/honeytoken alert fired", "capability":"alert", "required":true }, { "id":"query", "label":"Enumeration observed", "capability":"query", "required":true }, { "id":"dir", "label":"Directory activity on timeline","capability":"directory", "required":true }, { "id":"logon", "label":"Logon context captured", "capability":"logon", "required":false }] Uses Microsoft.Graph.Authentication + Graph Security (Secure Score / advanced hunting). EXPECTED RESULTS Report: a sensor-health summary, the current Identity Secure Score and top recommendations, and the delta versus the saved baseline. Scorecard: a per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. ISPM/Secure Score facts are indicative — verify current availability. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath = ".\MDIFULL-posture-baseline.json", [switch]$Scorecard, [string]$CriteriaPath, [string]$OutFile = 'MDI PoC Scorecard - Pilot DC - Validate.json', [int]$LookbackHours = 12)

How to run: Posture / coverage report (create or compare a baseline): powershell -ExecutionPolicy Bypass -File .\MDIFULL-Report.ps1 -SaveBaseline powershell -ExecutionPolicy Bypass -File .\MDIFULL-Report.ps1 PoC scorecard against your criteria: powershell -ExecutionPolicy Bypass -File .\MDIFULL-Report.ps1 -Scorecard ` -CriteriaPath .\criteria.json -OutFile "MDI PoC Scorecard - Pilot DC - Validate.json" Sample criteria.json: [{ "id":"alert", "label":"Recon/honeytoken alert fired", "capability":"alert",

"required":true }, { "id":"query", "label":"Enumeration observed", "capability":"query", "required":true }, { "id":"dir", "label":"Directory activity on timeline","capability":"directory", "required":true }, { "id":"logon", "label":"Logon context captured", "capability":"logon", "required":false }] Uses Microsoft.Graph.Authentication + Graph Security (Secure Score / advanced hunting). EXPECTED RESULTS Report: a sensor-health summary, the current Identity Secure Score and top recommendations, and the delta versus the saved baseline. Scorecard: a per-criterion PASS/FAIL table, an overall pass rate, and a saved scorecard file. .NOTES White-label. Read-only. ISPM/Secure Score facts are indicative — verify current availability. Whether the PoC "passed" and how criteria are weighted is the customer's decision. > [CmdletBinding()] param([switch]$SaveBaseline, [string]$BaselinePath = ".\MDIFULL-posture-baseline.json", [switch]$Scorecard, [string]$CriteriaPath, [string]$OutFile = 'MDI PoC Scorecard - Pilot DC - Validate.json', [int]$LookbackHours = 12)

MDIFULL-TestSignal.ps1

Purpose: Produces harmless, READ-ONLY identity signal so the MDI detection pipeline has something to catch during the "prove the value" phase. Run from a domain-joined TEST WORKSTATION signed in as a TEST ACCOUNT. It: 1. Runs benign directory enumeration (net group / net user / nltest / an LDAP read) — the same commands admins run daily. MDI flags the reconnaissance PATTERN; nothing is changed. 2. Touches the honeytoken decoy account with a benign lookup so that clean, high-fidelity signal is raised. NO attack tooling, credential dumping, exploitation, or AD modification is ever performed. -WhatIf shows what it would run without running it. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\MDIFULL-TestSignal.ps1 -WhatIf Generate benign signal (on the test workstation as the test account): powershell -ExecutionPolicy Bypass -File .\MDIFULL-TestSignal.ps1 -HoneytokenSam "svc-legacy-backup" EXPECTED RESULTS Reconnaissance and honeytoken detections appear in MDI for MDIFULL-Validate.ps1 -Check Signal and MDIFULL-Report.ps1 -Scorecard to confirm and score. .NOTES White-label. BENIGN, READ-ONLY signal only (enumeration + honeytoken lookup). Never real attacks. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$HoneytokenSam, [string]$Tag = 'MDI-TESTSIGNAL')

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\MDIFULL-TestSignal.ps1 -WhatIf Generate benign signal (on the test workstation as the test account): powershell -ExecutionPolicy Bypass -File .\MDIFULL-TestSignal.ps1 -HoneytokenSam "svc-legacy-backup" EXPECTED RESULTS Reconnaissance and honeytoken detections appear in MDI for MDIFULL-Validate.ps1 -Check Signal and MDIFULL-Report.ps1 -Scorecard to confirm and score. .NOTES White-label. BENIGN, READ-ONLY signal only (enumeration + honeytoken lookup). Never real attacks. > [CmdletBinding(SupportsShouldProcess = $true)] param([string]$HoneytokenSam, [string]$Tag = 'MDI-TESTSIGNAL')

MDIFULL-Validate.ps1

Purpose: Read-only validation for the "Microsoft Defender for Identity" solution. Confirms the piece named by -Check: the Directory Service Account, pilot sensor health, sensitive-entity tagging, honeytoken tagging, or the identity signal (that the Identity* tables stream and a controlled/benign test detection is visible). Maps to the Testing Guide gates for all three phases (Deploy, Enable, Prove). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDIFULL-Validate.ps1 -Check Sensor powershell -ExecutionPolicy Bypass -File .\MDIFULL-Validate.ps1 -Check Signal -HoneytokenSam 'svc-legacy-backup' -Check accepts: DSA | Sensor | Sensitive | Honeytoken | Signal | All (default All). Uses Microsoft.Graph.Authentication and the Microsoft 365 Defender / Graph Security APIs. Reads the MDI profile and runs read-only advanced hunting; makes no changes. EXPECTED RESULTS PASS/REVIEW

lines per check with the object names from the Implementation Guide, plus row counts per identity table and any recent honeytoken activity — so you can confirm each Testing Guide gate before expanding beyond the pilot domain controller. .NOTES White-label. Read-only. Some MDI settings are portal-only — where an API read is unavailable the script prints the exact portal path to confirm manually. > [CmdletBinding()] param([ValidateSet('DSA','Sensor','Sensitive','Honeytoken','Signal','All')] [string]$Check = 'All', [string]$HoneytokenSam = 'REPLACE-with-honeytoken-samAccountName', [int]$LookbackDays = 1)

How to run: powershell -ExecutionPolicy Bypass -File .\MDIFULL-Validate.ps1 -Check Sensor powershell -ExecutionPolicy Bypass -File .\MDIFULL-Validate.ps1 -Check Signal -HoneytokenSam 'svc-legacy-backup' -Check accepts: DSA | Sensor | Sensitive | Honeytoken | Signal | All (default All). Uses Microsoft.Graph.Authentication and the Microsoft 365 Defender / Graph Security APIs. Reads the MDI profile and runs read-only advanced hunting; makes no changes. EXPECTED RESULTS PASS/REVIEW lines per check with the object names from the Implementation Guide, plus row counts per identity table and any recent honeytoken activity — so you can confirm each Testing Guide gate before expanding beyond the pilot domain controller. .NOTES White-label. Read-only. Some MDI settings are portal-only — where an API read is unavailable the script prints the exact portal path to confirm manually. > [CmdletBinding()] param([ValidateSet('DSA','Sensor','Sensitive','Honeytoken','Signal','All')] [string]$Check = 'All', [string]$HoneytokenSam = 'REPLACE-with-honeytoken-samAccountName', [int]$LookbackDays = 1)